

(a) 1-2-3-4-5-6-7-8-9-10 (24)

(b) 1-2-3-4-5-6-7-8-9-10 30

(c) odd
even

(d) For a method that returns a value, it's to return the result of a calculation, like in (a).
For a method that return void, it does not return any value, just to implement a series of operation.

(i) range: [1, 6]

(ii) range: [1, 6]

(iii) Because for (int i = 0; i < 4; i++)
i has four possibilities: 0, 1, 2, 3

so rnd.nextInt(6) + 1 also has four possibilities

(iv) ① Improved code reusability ② code reliability
don't need to write the algorithm for a random number again nextInt()(b) (i) public class int roll() {
return rnd.nextInt(6) + 1;

```

    {
(ii) public static void main(String[] args) {
    for (int i = 0; i < 5; i++) {
        System.out.println(roll() + " ");
    }
}

```

(c) (i) range: [0, 1]

the if is testing for whether the random points in the square land inside the circle.

(ii) the cnt stands for the number of points in the circle, while N represent the total amount of points;

The area of the square is 1, the radius of circle is 0.5,

$$\frac{A_{\text{square}}}{A_{\text{circle}}} \approx \pi \times 0.5 \quad \pi \approx 4 \times \frac{\text{cnt}}{N}$$

(iii) cnt is very likely to be a decimal instead of an integer. Use 4.0 to improve the accuracy of the calculation and make the result more precise.

3.

(a)

```
public static int nextTerm(int n) {  
    if (n % 2 == 0) {  
        return n/2;  
    } else {  
        return 3*n + 1;  
}  
static nextTerm nxl = new nextTerm();
```

```
public static int nextTerm(int n) {  
    if (n % 2 == 0) {  
        return n/2;  
    } else {  
        return 3*n + 1;  
    }  
}
```

(b)

```
public static int steps(int n) {  
    int count = 0;  
    while (n != 1) {  
        n = nextTerm(n);  
        count++;  
    }  
    return count;  
}
```

(c)

(d) import java.util.Scanner;

```
public class Collatz {
```

```
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
        System.out.print("Enter a positive integer");  
        int n = scan.nextInt();
```

```
System.out.println();  
while (n != 1) {  
    n = nextTerm(n);  
    System.out.print(n + " ");  
}  
System.out.println("Steps: " + steps(n));  
}
```

Expected output:

For 6:

6 3 10 5 16 8 4 2

Steps: 8

For 7:

7 12 11 34 17 52 26 13 40 20 10 5 16 8 4 2

Steps: 16